

An I/O Lower Bound for Exact Attention

Samuel Mausberg

Independent Researcher

Abstract

Every bounded deterministic real-arithmetic program that computes exact softmax attention on $Q, K, V \in \mathbb{R}^{n \times d}$ requires $\Omega(nd + n^2/M)$ transfers between slow memory and an M -word cache, for $n \geq d^2$, $d \geq 2$, and $M \geq d^2$. The allowed operations are $+$, $-$, $\times$, $\div$, $\exp$; recomputation and finite branching are permitted. No pairwise intermediate value is prescribed. The proof adjoins the entries of each epoch's local Jacobian to a field that retains all earlier such entries. An epoch contributes at most $4M^2$ generators, whereas the output derivatives determine $n(n-1)$ algebraically independent kernel ratios. The bound is tight for each fixed d . We also show why field containment does not recover the uniform dependence on d . For powers of two $d \geq 8$ dividing n , a Strassen-based prefix reaches the entire attention coefficient field after $O(nd^{\sigma-1} + n^2d^{\sigma-2}/M)$ transfers, where $\sigma = \log_2 7$. This prefix can occur in a correct $O(n^2d)$ -work program, yet on a parameter sequence it costs $o(nd + n^2d/M)$. It computes an auxiliary vector, not attention. A separate numerical compression theorem gives $\Omega(nd + n^2d^2/M)$ when the epochs recover all pair scores or kernels; that recovery hypothesis is not part of the unrestricted lower bound.

1 Introduction

For $Q, K, V \in \mathbb{R}^{n \times d}$, exact attention is the map

$$W_{ij} = \exp(q_i \cdot k_j), \quad A_{ij} = \frac{W_{ij}}{\sum_{h=1}^n W_{ih}}, \quad Y = AV, \quad (1)$$

where q_i and k_j are rows. The inputs begin in slow memory, the output must be written there, and arithmetic operates on at most M fast real words. We count scalar loads and stores.

The blocking schedule underlying FlashAttention uses $O(nd + n^2 d^2 / M)$ transfers and $O(n^2 d)$ arithmetic work [DFE⁺22]. A formula with n^2 pairwise scores does not require a program to form those scores. Products of coordinate-wise exponentials already avoid them, and more general rearrangements can combine terms before an individual kernel entry is available. Our lower bound makes no assumption about these intermediate values.

Theorem 1.1 (Exact attention). *For $n \geq d^2$, $d \geq 2$, and $M \geq d^2$, every bounded deterministic program over $+, -, \times, \div, \exp$ in the model of Section 2 that computes equation (1) on all real inputs has worst-case transfer complexity*

$$\text{IO} \geq \frac{1}{32} \left(nd + \frac{n^2}{M} \right). \quad (2)$$

The program may recompute values, branch on sign comparisons, and use fast matrix multiplication. It needs only a finite worst-case instruction bound for each fixed (n, d, M) .

For each fixed d , this matches the blocking upper bound, with comparison constants depending on that dimension. Uniformly in d , the bounds remain

$$\Omega\left(nd + \frac{n^2}{M}\right) \leq \text{IO}^*(n, d, M) \leq O\left(nd + \frac{n^2 d^2}{M}\right), \quad (3)$$

where IO^* is the least worst-case cost among exact families with $O(n^2 d)$ work. The lower bound applies more broadly, as the theorem states. We do not prove even the intermediate $\Omega(nd + n^2 d / M)$ bound when d grows.

The proof follows derivatives through the memory schedule. Since Y is linear in V , its Jacobian contains every entry of A . The ratios A_{ij}/A_{in} are $n(n-1)$ algebraically independent functions over the rational input field. An epoch of at most M transfers has at most $2M$ incoming and $2M$ outgoing values, so its local Jacobian has at most $4M^2$ entries. The chain rule puts all output derivatives in the field generated by these local entries throughout the execution. This requires $\Omega(n^2/M^2)$ epochs when the nontrivial term dominates. Derivatives are proof objects; the program never has to compute them.

Using field containment as the sole completion condition does not recover the dimension factor. Write

$$F_0 = \mathbb{R}(Q, K, V), \quad R_{ij} = \exp(q_i \cdot (k_j - k_n)) \quad (j < n), \quad \mathcal{T} = F_0(R_{ij} : i \in [n], j < n). \quad (4)$$

For powers of two $d \geq 8$ dividing n , we construct a prefix whose local-Jacobian history already contains $\mathcal{T}$, at a cost of

$$O\left(nd^{\sigma-1} + \frac{n^2 d^{\sigma-2}}{M}\right), \quad \sigma = \log_2 7. \quad (5)$$

It uses Strassen's bilinear decomposition to compute weighted sums of scalar exponentials. Differentiation separates the summands in the function field, where integer products recover the original kernels. Those products are not evaluated by the prefix. At $n = d^3$ and $M = d^2$, its cost is $\Theta(d^{\sigma+2}) = o(d^5)$, while $nd + n^2 d / M = \Theta(d^5)$. Appending a correct attention computation gives a uniform $O(n^2 d)$ -work program

with the same cheap prefix. Thus containing the coefficient field cannot, on its own, certify that the larger transfer cost has already been paid.

Numerical recovery imposes a different requirement. If m differentiable real summaries can decode B selected pair scores or kernels, then $B \leq m^2/d^2 + 2m$. Appendix B proves this compression bound and the full attention lower bound under pair coverage. It is a conditional statement about numerical values, not a consequence of field containment.

Prior work. Hong and Kung’s red-blue pebble game relates I/O to small-boundary partitions of a specified computation graph [HK81, Theorem 3.1]. We partition the executed program, without prescribing its graph. Saha and Ye permit fast matrix multiplication but explicitly assume that the program computes $QK^\top$, though it need not store that matrix [SY24, Section 4]. With $\mathbb{F}_q$ words and $q > n$, their Theorem 4.8 gives $\Omega(\min\{n^2d^2/M, n^2\})$ transfers. These numbers refer to arXiv:2402.07443v1. The present model instead allows arbitrary real words and analytic arithmetic, but no digit extraction or real-to-integer conversion. Their bound is stronger in d but applies to programs that compute $QK^\top$. To our knowledge, Theorem 1.1 is the first nontrivial I/O lower bound for exact attention that does not prescribe the computation of scores or kernels. The blocking comparison is the upper bound of [DFE⁺22, Theorem 2]; Appendix C supplies a schedule for the full parameter range in equation (3).

Abelson used derivatives to lower-bound communication between processors [Abe80]. The numerical factorisation argument in Lemma 2.2 uses this chain-rule principle. Here we count generators contributed by local Jacobians throughout an execution. The extended Baur–Strassen theorem of Saha, Xu, Ye, and Yu bounds differentiation in arithmetic circuits with exponentials and logarithms [SXY26, Theorem 4.4]; that circuit-size statement does not by itself preserve an I/O schedule. Our argument uses no numerical differentiation algorithm.

2 The machine and its open execution paths

For each integer triple (n, d, M) , a machine has M fast real registers and an unbounded array of slow real words. The input matrices occupy designated slow addresses. Output addresses are separate and initially unwritten. Initial register values and program constants are independent of the inputs. Loading or storing one scalar costs one transfer. Arithmetic acts only on fast registers and may overwrite an operand.

The scalar instructions are $+$, $-$, $\times$, $\div$, $\exp$, copies , and sign comparisons of register values or their differences. A division must have a nonzero denominator on its executed branch. Addresses use discrete counters and finite control. There is no instruction that converts an arbitrary real value to an integer, extracts a digit, performs a real-indexed lookup, or evaluates a function purportedly encoded in a real word. Code constants do not provide such operations. Recomputation and fast matrix multiplication are permitted.

Programs are deterministic and have a finite worst-case instruction bound at each fixed parameter triple. All instructions, including comparisons and counter updates, count as work. A finite I/O claim also has a finite worst-case transfer bound. Thus the complete computation can be unrolled into a finite decision tree. The lower bounds allow any finite work bound and arbitrary input-independent real constants. Algorithmic upper bounds use $O(n^2d)$ work with a constant uniform in the parameters. All complexity statements are worst-case statements on arbitrary real inputs. The same conventions apply to a general finite input vector x and output vector $f(x)$.

Lemma 2.1 (An open execution path). *A bounded program defined on a nonempty open input set has one execution path followed on a nonempty open subset. On that subset, all values appearing in the path are analytic functions of the inputs, and the accessed addresses are fixed.*

Proof. Start with a connected open subset. Arithmetic preserves analyticity where the denominators of executed divisions are nonzero. At a sign test, the analytic test function either vanishes identically or is nonzero on some smaller open subset, on which one sign is fixed. Choose the corresponding branch. Counter values and addresses are then fixed along the path. Repeating this step reaches a leaf because the decision tree is finite. The resulting nonempty open set may be shrunk to be connected. $\square$

Fix such a connected open set U . Analytic functions on U form an integral domain; we work in its fraction field. The rational function field $\mathbb{R}(x_1, \dots, x_p)$ embeds in this field because no nonzero polynomial vanishes on an open set. Statements about algebraic independence below concern functions on U , rather than particular real numbers. Poles introduced by a field identity have no implication for the instructions executed by the program.

For an analytic vector $f(x)$, define

$$\tau_1(f) = \text{trdeg}_{\mathbb{R}(x)} \mathbb{R}(x)(Df). \quad (6)$$

The right side is the transcendence degree of the field generated by the scalar entries of the Jacobian. It can exceed the number of input coordinates. Its numerical rank cannot exceed the number of inputs.

Lemma 2.2 (Numerical factorisation). *Suppose $C : U \rightarrow \mathbb{R}^m$ and D are continuously differentiable, with D defined on a neighbourhood of $C(U)$. If $F = D \circ C$, then $\text{rank } DF(x) \leq m$ on U . In particular, fewer than p such numerical summaries cannot encode and decode all p independent real input coordinates on an open set.*

Proof. The chain rule gives $DF(x) = DD(C(x)) DC(x)$. Its rank is at most m . Apply this to $F(x) = x$ for the last statement. $\square$

On an open execution path, this rules out lossless packing with numerical differentiable decoding. Field membership supplies no such decoder: it is an identity between functions, not an instruction for recovering their numerical values.

3 The derivative field of an execution

An epoch is a consecutive part of an execution with at most M transfers. Partition a path with I transfers into e epochs, using M transfers per epoch except possibly the last. Arithmetic before the first transfer and after the last is included. A path with no transfers has one epoch. In all cases,

$$e \leq I/M + 1. \quad (7)$$

Lemma 3.1 (Boundary factorisation). *For each epoch t , there are incoming values $C_t(x) \in \mathbb{R}^{a_t}$ and outgoing values $B_t(x) \in \mathbb{R}^{b_t}$, with $a_t, b_t \leq 2M$, and an analytic map Φ_t on a neighbourhood of a realised boundary point such that*

$$B_t(x) = \Phi_t(C_t(x)). \quad (8)$$

Every newly stored slow word and every final cache value occurs in B_t . Incoming values are initial cache values or versions of slow words that existed before the epoch.

Proof. Trace the epoch with its addresses fixed. Its initial cache contributes at most M arguments. For a load from a word whose current version predates the epoch, add its value as an external argument. There are at most M such loads. A load that follows a same-epoch store is replaced by the expression for that stored version, including through any intervening overwrites. It introduces no external argument.

Every instruction is now an expression in these arguments. Record all store values, at most M , and the final cache, at most M , as the outgoing vector. At any realised boundary point the original executed denominators

are nonzero. They remain nonzero in a sufficiently small neighbourhood of that point when the incoming arguments are treated as independent. The resulting expressions define Φ_t there. Shrinking the input open set for the finitely many epochs gives the claimed local factorisations. They need not be unique off the realised set of boundary values. $\square$

As illustrated in Figure 1, use the maps supplied by the executed expressions in Lemma 3.1 and set

$$\Lambda_t(x) = D\Phi_t(C_t(x)), \quad \mathcal{H}_0 = F_0 = \mathbb{R}(x), \quad \mathcal{H}_t = \mathcal{H}_{t-1}(\Lambda_t). \quad (9)$$

Parentheses mean adjoining all scalar entries. These are ordinary fields. No further differentiation is implicit in their definition.

Theorem 3.2 (I/O and the first-derivative field). *Let a bounded analytic program compute $f(x)$ on a nonempty open set, using M fast words and I transfers on an open execution path. Then*

$$\tau_1(f) \leq \sum_{t=1}^e a_t b_t \leq 4M^2 e, \quad I \geq \frac{\tau_1(f)}{4M} - M. \quad (10)$$

Proof. The induction tracks every extant word in both memories. A later load therefore reuses the derivatives of a stored word rather than introducing a new input function.

Initially, first derivatives of input words and constants are 0 or 1, hence belong to F_0 . Suppose that every first derivative of every extant word after epoch $t-1$ belongs to $\mathcal{H}_{t-1}$. By Lemma 3.1, this holds for all entries of DC_t . Differentiating equation (8) gives

$$DB_t = \Lambda_t DC_t. \quad (11)$$

Every scalar entry on the right belongs to $\mathcal{H}_t$. Newly stored words and final cache values occur in B_t ; an unchanged slow word keeps its previous derivatives. Overwritten versions need not be kept as numerical words, although their previously adjoined functions remain in the history field. The induction follows.

All output derivatives therefore belong to $\mathcal{H}_e$. Adjoining $a_t b_t$ elements increases transcendence degree by at most $a_t b_t$, regardless of any algebraic dependencies among them. Consequently

$$\tau_1(f) \leq \text{trdeg}_{F_0} \mathcal{H}_e \leq \sum_{t=1}^e a_t b_t \leq 4M^2 e.$$

Substitute equation (7) and rearrange. $\square$

A reload with no further arithmetic has an identity local map and contributes no new generator. A long computation inside an epoch may produce many exponentials, but only the entries of the epoch's boundary Jacobian are adjoined in equation (9). Closure under further differentiation would give a different and invalid charge. For example, $H_L(a, x) = \sum_{\ell=1}^L e^{a^\ell x}$ uses two loads, one store, and a constant-size cache. Its successive x -derivatives recover all L exponentials by a Vandermonde system, but its first-derivative field has only two generators. Thus an epoch may not be charged for every derivative of a cached function.

4 The attention coefficient field

Lemma 4.1 (Exponentials of polynomials). *Let $p_1, \dots, p_s \in \mathbb{R}[x]$. Suppose every nonzero integer linear combination of them is nonconstant. Then $e^{p_1}, \dots, e^{p_s}$ are algebraically independent over $\mathbb{R}(x)$ on every nonempty open set.*

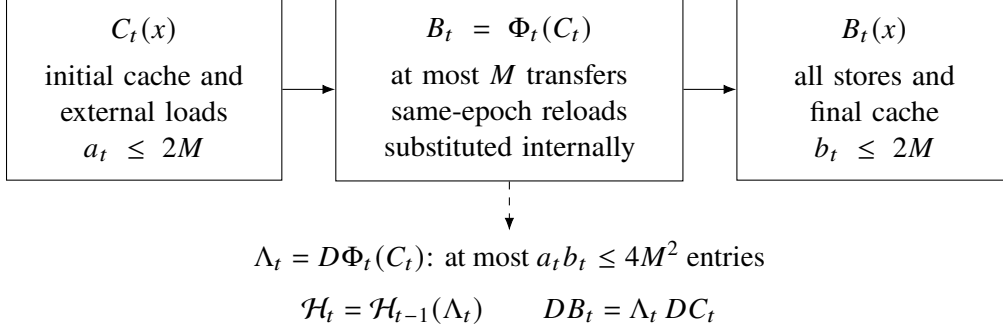


Figure 1: One epoch and its local Jacobian. The history retains the entries of Λ_t as functions, without making their numerical values available to the machine. A reloaded word inherits its derivatives from earlier epochs.

Proof. Clear denominators in a putative polynomial relation among the exponentials and collect monomials to obtain

$$\sum_{a \in S} r_a(x) e^{P_a(x)} = 0, \quad P_a = \sum_{j=1}^s a_j p_j, \quad (12)$$

where S is a finite set of distinct nonnegative integer vectors and each r_a is a nonzero polynomial. The analytic identity extends from an open set to all of $\mathbb{R}^P$. For a generic vector z , restriction to $x = uz$ keeps every $r_a(uz)$ nonzero as a polynomial in u and makes every difference $P_a(uz) - P_b(uz)$ nonconstant. Such z exists: each excluded condition forces a nonzero homogeneous component of one of finitely many polynomials to vanish.

For sufficiently large positive u , the distinct exponent polynomials are strictly ordered. Divide equation (12) by the largest exponential. Each remaining term is a polynomial times $e^{-q(u)}$, where $q(u) \rightarrow +\infty$ is a nonconstant polynomial. These terms tend to zero faster than every inverse power of u . The remaining coefficient is a nonzero polynomial and cannot do so. $\square$

Proposition 4.2 (The coefficient field). *For $n \geq 2$ and $d \geq 1$, the functions R_{ij} of equation (4) are algebraically independent over F_0 , and*

$$F_0(DY) = \mathcal{T}, \quad \tau_1(Y) = n(n-1). \quad (13)$$

Proof. The polynomial $q_i \cdot (k_j - k_n)$ has coefficient one on $Q_{i1}K_{j1}$ for $j < n$. No other polynomial in this family contains that monomial. Their nonzero integer combinations are therefore nonconstant, and Lemma 4.1 applies.

Linearity in V gives $\partial Y_{i1}/\partial V_{j1} = A_{ij}$. Thus $R_{ij} = A_{ij}/A_{in}$ belongs to $F_0(DY)$, proving one containment. Conversely, set $R_{in} = 1$. The entries of Y are rational functions of R and V , with denominator $1 + \sum_{j < n} R_{ij}$. Every input derivative of R_{ij} is a polynomial times R_{ij} . The field $\mathcal{T}$ is consequently closed under input differentiation, and all entries of DY belong to it. Independence gives its transcendence degree. $\square$

Proof of Theorem 1.1. Choose an open execution path using Lemma 2.1. The function computed there is still attention, so Theorem 3.2 and Proposition 4.2 give

$$I \geq \frac{n(n-1)}{4M} - M.$$

There are also nd compulsory output stores on this path. If $M \leq n/4$, then $n(n-1) \geq n^2/2$ and $I \geq n^2/(16M)$. Combining this with $I \geq nd$ yields equation (2). If $M > n/4$, then $n^2/M < 4n \leq 2nd$, so the output stores alone suffice. The worst-case cost is at least the cost of this path. $\square$

5 A prefix obtained from a bilinear decomposition

We construct a cheap prefix whose derivative field contains $\mathcal{T}$. The numerical output consists of weighted sums of scalar exponentials. The kernels are recovered only as identities in a field.

We use the following recursive form of Strassen's identity [Str69]. The proof also establishes the independence property needed for the derivative field.

Lemma 5.1 (The seven-product decomposition). *Let $d = 2^k$ and $R = 7^k = d^\sigma$, where $\sigma = \log_2 7$. There are nonzero integer linear forms ℓ_r, m_r on $d \times d$ matrices and integers $c_{\alpha\beta r}$ such that*

$$(UZ^\top)_{\alpha\beta} = \sum_{r=1}^R c_{\alpha\beta r} \ell_r(U) m_r(Z). \quad (14)$$

All R forms on either input can be evaluated in $O(R)$ scalar instructions, with an $O(R)$ -transfer implementation using a constant number of fast words. The R bilinear polynomials $\ell_r(U) m_r(Z)$ are linearly independent. The forms ℓ_r span the dual of the d^2 -dimensional input space.

Proof. The seven-product identity [Str69], written in Appendix A, gives equation (14) by recursion. Its coefficients are integers. Evaluating all leaf forms satisfies $L(d) \leq 7L(d/2) + O(d^2)$, with $L(1) = O(1)$, hence $L(d) = O(R)$. Store every intermediate in slow memory and use a constant number of registers for each scalar instruction to obtain the stated transfer bound.

At $d = 2$, the coefficient vectors of the seven bilinear polynomials have a 7×7 minor of determinant one; the minor is displayed in Appendix A. Recursive leaf coefficient vectors are tensor products of these seven vectors, up to a permutation of monomials. They are linearly independent at every depth. Finally, if all $\ell_r(U)$ vanish, then equation (14) gives $UZ^\top = 0$ for every Z . Taking Z to be the identity gives $U = 0$. Thus the forms ℓ_r span the full dual space. $\square$

Assume $d \mid n$ and write $B = n/d$. Partition the rows of Q and K into B blocks of d rows, denoted $Q^{(a)}$ and $K^{(b)}$. For $a, b \in [B]$ and $r \in [R]$, define

$$u_{ar} = \ell_r(Q^{(a)}), \quad z_{br} = m_r(K^{(b)}), \quad E_{abr} = e^{u_{ar} z_{br}}, \quad t_b = V_{(b-1)d+1,1}. \quad (15)$$

The auxiliary vector has BR entries:

$$g_{ar} = \sum_{b=1}^B E_{abr} t_b. \quad (16)$$

All original input coordinates remain free. In particular, a scalar form u_{ar} can depend on many entries of its full matrix block.

Theorem 5.2 (Cost and derivative field of the auxiliary vector). *Let $d \geq 8$ be a power of two, $d \mid n$, $n \geq d^2$, and $M \geq d^2$. Computing the vector g of equation (16), with its BR outputs in designated slow addresses, has I/O complexity*

$$\Theta\left(BR + \frac{B^2 R}{M}\right) = \Theta\left(nd^{\sigma-1} + \frac{n^2 d^{\sigma-2}}{M}\right). \quad (17)$$

The upper bound has $O(BR + B^2 R) \subseteq O(n^2 d)$ work. Moreover,

$$F_0(Dg) = F_0(E_{abr} : a, b \in [B], r \in [R]) \supseteq \mathcal{T}, \quad \tau_1(g) = B^2 R. \quad (18)$$

For any epoch partition of this prefix, its history at completion contains $\mathcal{T}$.

Proof. Evaluate and store all u_{ar} and z_{br} using Lemma 5.1. This costs $O(BR)$ work and transfers. For a fixed r , divide the query and key indices into tiles of size at most

$$s = \min\{B, \lfloor M/8 \rfloor\}.$$

On a tile $I \times J$, load the current accumulators a_i , query scalars u_{ir} , key scalars z_{jr} , and markers t_j , and compute

$$b_i = a_i + \sum_{j \in J} e^{u_{ir} z_{jr}} t_j \quad (i \in I). \quad (19)$$

At most $4s + 4 \leq M$ fast words suffice. The tile uses $3|I| + 2|J| \leq 5s \leq M$ transfers, including the accumulator stores. Initialising the B accumulators and visiting all tiles costs at most

$$B + 5B \lceil B/s \rceil = O(B + B^2/M)$$

transfers for this r . The total over r is the upper bound in equation (17). There are $B^2 R$ scalar exponential terms and $O(BR)$ preprocessing instructions. Since $R = d^\sigma \leq d^3$, both are within a uniform $O(n^2 d)$ budget.

Fix r and choose a key coordinate $\kappa_r \in [d] \times [d]$ whose coefficient β_r in m_r is nonzero. The markers are independent of Q, K . Therefore

$$\frac{\partial g_{ar}}{\partial K_{\kappa_r}^{(b)}} = u_{ar} \beta_r E_{abr} t_b, \quad E_{abr} = \frac{\partial g_{ar} / \partial K_{\kappa_r}^{(b)}}{u_{ar} \beta_r t_b} \quad \text{in the function field.} \quad (20)$$

The denominator is a nonzero rational function, although it can vanish on particular inputs. The prefix performs no division by it. Every derivative of g is rational in the E_{abr} and the inputs, which proves the field equality in equation (18).

For a row $i = (a - 1)d + \alpha$ and $j = (b - 1)d + \beta$, equation (14) gives

$$W_{ij} = \prod_{r=1}^R E_{abr}^{c_{a\beta r}}. \quad (21)$$

All powers are integers, so equation (21) belongs to the ordinary field $F_0(E_{abr} : a, b \in [B], r \in [R])$. Taking W_{ij}/W_{in} gives $\mathcal{T} \subseteq F_0(E_{abr} : a, b \in [B], r \in [R])$.

For a fixed pair of blocks (a, b) , the exponent polynomials $u_{ar} z_{br}$ are linearly independent by Lemma 5.1. Different block pairs use disjoint sets of query–key monomials. Thus all $B^2 R$ exponent polynomials are linearly independent and homogeneous of degree two. Lemma 4.1 proves algebraic independence of the corresponding exponentials. Hence $\tau_1(g) = B^2 R$.

The lower bound now follows from Theorem 3.2 and the BR compulsory output stores. If $M \leq B\sqrt{R}/4$, then $I \geq B^2 R/(4M) - M \geq 3B^2 R/(16M)$. If $M > B\sqrt{R}/4$, then $B^2 R/M < 4B\sqrt{R} \leq 4BR$, which is controlled by the output stores. Combining the bounds proves equation (17). Finally, the induction in Theorem 3.2 puts $F_0(Dg)$ in the history at the end of any partition of this executed prefix. $\square$

Corollary 5.3. *Along $d = 2^k \geq 8$, $n = d^3$, and $M = d^2$, the prefix of Theorem 5.2 has $O(n^2 d)$ work and $\Theta(d^{\sigma+2}) = o(nd + n^2 d/M)$ transfers, while its history contains $\mathcal{T}$.*

Proof. Here $B = d^2$, $R = d^\sigma$, and both terms of equation (17) are $d^{\sigma+2}$. In contrast, $nd + n^2 d/M = d^4 + d^5$. Since $\sigma < 3$, their ratio tends to zero. $\square$

6 What field containment cannot certify

The prefix of Theorem 5.2 can be followed by the blocking attention algorithm while retaining the original inputs in slow memory. This gives a correct $O(n^2d)$ -work program. The continuation may ignore the auxiliary vector, so the argument concerns a completion condition stated solely in terms of the history field.

Theorem 6.1 (Field-containment certificates). *There are no absolute constants $c, C > 0$ and charges Π_t , defined for every epoch history of every correct $O(n^2d)$ -work attention program, with the following properties: $\Pi_0 = 0$; a prefix history containing $\mathcal{T}$ has charge at least $c n(n-1)$; and every epoch of at most M transfers satisfies*

$$\Pi_t - \Pi_{t-1} \leq C \left(M + \frac{M^2}{d} \right). \quad (22)$$

Charges may depend on the entire execution history and need not be monotone.

Proof. Take $n = d^3$, $M = d^2$, and $d = 2^k \geq 8$. Run the prefix in Theorem 5.2, then compute attention. Pad the end of the prefix with fewer than M transfers of constants so that it is an epoch boundary. On other parameter triples use the blocking algorithm; the resulting family has a uniform $O(n^2d)$ work bound.

By Corollary 5.3, the prefix uses $O(d^\sigma)$ epochs and its history contains $\mathcal{T}$. Summing equation (22) from zero gives

$$\Pi_t \leq O(d^\sigma)(d^2 + d^3) = O(d^{\sigma+3}) = o(d^6).$$

The completion condition instead requires $\Pi_t \geq c d^3(d^3 - 1) = \Omega(d^6)$, a contradiction. The argument uses only upper bounds on increments. $\square$

The same example excludes the smaller epoch capacity $O(M + M^2/d^2)$. More generally, field containment alone cannot certify a transfer cost above equation (5) on this parameter sequence. The additive $nd^{\sigma-1}$ term cannot be dropped unless it is absorbed, for example when $n \geq Md$.

This is a limit on the stated completion condition, not on all lower-bound methods. In particular, it does not address a charge that requires numerical production of the output or exploits a specific continuation. Nor does the prefix give an improved attention algorithm. The uniform gap in equation (3) remains.

A The bilinear identity and its independent factors

For 2×2 matrices $U = (u_{ij})$ and $B = (b_{ij})$, use the seven products

$$\begin{aligned} p_1 &= (u_{11} + u_{22})(b_{11} + b_{22}), & p_2 &= (u_{21} + u_{22})b_{11}, \\ p_3 &= u_{11}(b_{12} - b_{22}), & p_4 &= u_{22}(b_{21} - b_{11}), \\ p_5 &= (u_{11} + u_{12})b_{22}, & p_6 &= (u_{21} - u_{11})(b_{11} + b_{12}), \\ p_7 &= (u_{12} - u_{22})(b_{21} + b_{22}). \end{aligned}$$

Strassen's identity [Str69] is

$$UB = \begin{pmatrix} p_1 + p_4 - p_5 + p_7 & p_3 + p_5 \\ p_2 + p_4 & p_1 - p_2 + p_3 + p_6 \end{pmatrix}. \quad (23)$$

Expanding its four entries verifies the identity over the integers. Substitute $B = Z^\top$ to obtain the form used in the main text. Applying the same identity recursively to matrix blocks gives 7^k leaf products for $d = 2^k$. Each leaf is an integer linear form in U times an integer linear form in Z ; each output is an integer linear combination of these products.

Order the four entries of U and B row by row, and order their sixteen pairwise monomials first by the U entry and then by the B entry. The coefficient matrix of $(p_1, \dots, p_7)$ has, at zero-based column indices 0, 1, 3, 6, 8, 9, 12, the minor

$$\begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ -1 & -1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}, \quad \det = 1. \quad (24)$$

This proves independence at the base. At the next level, a leaf index is a pair of leaf indices; its coefficient on a query–key monomial is the product of the corresponding coefficients at the two levels. After reordering the monomial coordinates, the coefficient vectors are therefore the tensor products of the base vectors. Tensor products of independent vectors are independent: extend the base vectors to a basis and apply the dual basis in each tensor factor. Induction proves independence of the 7^k bilinear polynomials. Transposing the key matrix merely permutes the monomials.

The same recursion evaluates all input linear forms with $L(d) \leq 7L(d/2) + cd^2$ instructions. No product of the two inputs is needed for this preprocessing. Copies of subblocks and sums or differences require $O(d^2)$ instructions at a node. The recurrence sums to $O(d^\sigma)$; storing each intermediate in slow memory gives at most a constant number of transfers per instruction. This implementation deliberately makes no claim of optimal I/O for the linear transforms.

B Numerical recovery of pairs

This appendix gives a real-word counterpart of the matrix compression step in [SY24, Lemma 4.6], with a hypothesis absent from Theorem 1.1. A vector of summaries numerically recovers a target vector when there is a continuously differentiable decoder from those numerical values alone, on a neighbourhood of their image. The decoder may use input-independent constants. It is not given arbitrary elements of the rational input field.

Lemma B.1 (Rank of a selected dot-product family). *Let E be a bipartite set of B selected pairs between independent query and key vectors in $\mathbb{R}^d$. If ρ is the generic Jacobian rank of $(q_i \cdot k_j)_{(i,j) \in E}$, then*

$$B \leq \frac{\rho^2}{d^2} + 2\rho. \quad (25)$$

Proof. Let

$$r = \sum_i \min(\deg(i), d), \quad c = \sum_j \min(\deg(j), d).$$

Choose the keys in generic position so that every set of at most d of them is independent; distinct Vandermonde vectors provide a witness. The derivatives in the query coordinates form blocks on disjoint output coordinates, with total rank r . Thus $\rho \geq r$. Interchanging the roles gives $\rho \geq c$. These generic inequalities hold simultaneously, since the witnessing nonzero minors are polynomials.

There are at most r/d query vertices and c/d key vertices with degree at least d . At most rc/d^2 edges join two such vertices. Edges incident to a lower-degree query number at most r , and the remaining edges incident to a lower-degree key number at most c . Hence $B \leq rc/d^2 + r + c \leq \rho^2/d^2 + 2\rho$. $\square$

Theorem B.2 (Numerical compression). *Suppose m continuously differentiable real summaries numerically recover B selected entries from one of the families*

$$q_i \cdot k_j, \quad e^{q_i \cdot k_j}, \quad e^{q_i \cdot (k_j - k_n)} \quad (j < n)$$

on a nonempty open input set. Then

$$B \leq \frac{m^2}{d^2} + 2m. \quad (26)$$

The summaries may depend on V and on every input coordinate.

Proof. Numerical factorisation bounds the rank of the recovered vector by m , by Lemma 2.2. Apply Lemma B.1 for scores. Coordinate-wise exponentiation multiplies its Jacobian by an invertible diagonal matrix, preserving rank. For the centred family, change variables from K to $(K_1 - K_n, \dots, K_{n-1} - K_n, K_n)$. This is an invertible linear change, and the centred key vectors are independent coordinates. The selected minors are nonzero on a dense open subset, so the argument applies inside the stated open input set. $\square$

Corollary B.3 (The full bound under pair coverage). *In the attention parameter range, suppose the epochs of an open execution path numerically recover, between them, every entry of one full family in Theorem B.2. The path has $\Omega(nd + n^2 d^2 / M)$ transfers, with an absolute constant.*

Proof. Every value computed during an epoch is an analytic function of its at most $2M$ incoming values. By Theorem B.2, one epoch recovers at most $4M^2/d^2 + 4M \leq 8M^2/d^2$ selected pairs. There are at least $n(n-1)$ distinct targets in a full family. Together with equation (7), this gives

$$I \geq \frac{n(n-1)d^2}{8M} - M.$$

If $M \leq nd/8$, the right side is at least a constant times $n^2 d^2 / M$. Otherwise this latter term is at most $8nd$. The nd output stores complete the bound in both cases. $\square$

A program can compute W_{ij} as $\prod_{\ell=1}^d e^{Q_{i\ell} K_{j\ell}}$, without forming the sum $q_i \cdot k_j$. Its completed kernels satisfy the hypothesis of Corollary B.3. This example prevents interpreting pair coverage as an explicit-score assumption. No coverage property is proved for every exact program.

C The blocking upper bound and small parameter cases

We include the schedule only to specify the uniform comparison in equation (3); the bound is the established attention tiling bound [DFE⁺22, SY24]. For $M \geq 16d$, choose

$$b = \min\{n, \lfloor M/(8d) \rfloor\}.$$

Keep b query rows, b length- d numerator accumulators, and b denominators in fast memory. Scan the key-value rows. For each such row, compute the b exponential weights and update the accumulators. All denominators are positive. Divide the numerators and write the output block, then proceed to the next query block.

The query block and numerators use $2bd$ words. Denominators use b , a key and value row use $2d$, and eight scratch words suffice for the scalar inner loops. For $d \geq 2$ and $M \geq 16d$,

$$2bd + b + 2d + 8 \leq M.$$

The transfers are $O(nd + \lceil n/b \rceil nd)$, hence $O(nd + n^2 d^2 / M)$. Every pair is processed with $O(d)$ work. A product of coordinate-wise exponentials can replace the dot-product sum with the same asymptotic work and transfer count.

If $d^2 \leq M < 16d$, then $d < 16$ and $M < 256$. A scalar implementation can spill all nonresident partial sums and evaluate the attention formula with four fast words and $O(n^2 d + nd)$ transfers. For example, one register holds the current sum, two hold the next input operands, and one holds a temporary product or exponential; completed values and outer accumulators may be spilled before the next step. Since $M/d < 16$, this count is within an absolute constant of $nd + n^2 d^2 / M$. The work remains $O(n^2 d)$. This covers the bounded parameter cases as well as the regime in which full rows fit.

AI methodology

This paper was developed with AI assistance from GPT-6 Astra Pro and Claude models.

References

- [Abe80] Harold Abelson. Lower bounds on information transfer in distributed computations. *Journal of the ACM*, 27(2):384–392, 1980. doi:10.1145/322186.322200.
- [DFE⁺22] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc., 2022. doi:10.52202/068431-1189.
- [HK81] Jia-Wei Hong and H. T. Kung. I/O complexity: The red-blue pebble game. In *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing*, pages 326–333. ACM, 1981. doi:10.1145/800076.802486.
- [Str69] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969. doi:10.1007/BF02165411.
- [SXYY26] Barna Saha, Yinzhan Xu, Christopher Ye, and Hantao Yu. On the computational hardness of transformers. In *Proceedings of the 58th Annual ACM Symposium on Theory of Computing*, pages 1073–1084. ACM, 2026. arXiv:2603.11332v1. doi:10.1145/3798129.3800819.
- [SY24] Barna Saha and Christopher Ye. I/O complexity of attention, or how optimal is FlashAttention? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 43024–43042. PMLR, 2024. arXiv:2402.07443v1. URL: <https://proceedings.mlr.press/v235/saha24a.html>.